

2520090198_Siri Chandana_OS_PRACTICAL_8

Task 1 - Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
GNU nano 8.7.1 memory_allocation.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc()
    int *a = (int *)malloc(3 * sizeof(int));
    if (a == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Memory allocated using malloc()\n");

    for (i = 0; i < 3; i++)
    {
        a[i] = i + 1;
    }

    printf("malloc values: ");
    for (i = 0; i < 3; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");

    // calloc()
    int *b = (int *)calloc(3, sizeof(int));
    if (b == NULL)
    {
        printf("Memory allocation failed\n");
        free(a);
        return 1;
    }

    printf("Memory allocated using calloc()\n");

    printf("calloc values: ");
    for (i = 0; i < 3; i++)
    {
        printf("%d ", b[i]);
    }
    printf("\n");

    // realloc()
    a = (int *)realloc(a, 5 * sizeof(int));
    if (a == NULL)
    {
        printf("Reallocation failed\n");
        free(b);
        return 1;
    }

    a[3] = 4;
    a[4] = 5;

    printf("Memory reallocated using realloc()\n");

    printf("realloc values: ");
    for (i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");

    // free()
    free(a);
    free(b);

    printf("Memory freed using free()\n");

    return 0;
}
```

```
sirichandana@siri:~/practical_memory$ cd ~/practical_memory
sirichandana@siri:~/practical_memory$ nano memory_allocation.c
sirichandana@siri:~/practical_memory$ gcc memory_allocation.c -o memory_allocation
sirichandana@siri:~/practical_memory$ ./memory_allocation
Memory allocated using malloc()
malloc values: 1 2 3 4 5
Memory allocated using calloc()
calloc values: 0 0 0 0 0
Memory reallocated using realloc()
realloc values: 1 2 3 4 5
Memory freed using free()
sirichandana@siri:~/practical_memory$ █
```

Task 2 - Develop a program that demonstrates Copy-on-Write behavior after a `fork()`.

Analyze memory usage before and after modifying data in the child process.

```

GNU nano 8.7.1                                     copy_on_write.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main()
{
    int *data;

    // Allocate memory
    data = (int *)malloc(SIZE * sizeof(int));

    if (data == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    // Initialize the memory
    for (int i = 0; i < SIZE; i++)
    {
        data[i] = i;
    }

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated and initialized.\n");
    printf("Address of data: %p\n", (void *)data);

    printf("\nCreating child process using fork()...\n");

    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        free(data);
        return 1;
    }

    else if (pid == 0)
    {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Address of data in child: %p\n", (void *)data);

        printf("Before modification: data[0] = %d\n", data[0]);

        // Modify the data
        data[0] = 9999;

        printf("After modification: data[0] = %d\n", data[0]);
        printf("Child modified the memory.\n");
        printf("Copy-on-Write creates a private copy of the modified page.\n");

        sleep(5);

        free(data);
        exit(0);
    }

    else
    {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent data[0] = %d\n", data[0]);

        sleep(2);

        printf("After child modification, parent data[0] = %d\n", data[0]);
        printf("Parent still has its original data.\n");

        wait(NULL);

        free(data);
    }

    return 0;
}

```

```
sirichandana@siri:~/practical_memory$ cd ~/practical_memory
sirichandana@siri:~/practical_memory$ nano copy_on_write.c
sirichandana@siri:~/practical_memory$ gcc copy_on_write.c -o copy_on_write
sirichandana@siri:~/practical_memory$ ./copy_on_write
Parent PID: 1714
Memory allocated and initialized.
Address of data: 0x70bd5b02f010

Creating child process using fork()...

Parent Process
Parent PID: 1714
Child PID: 1715
Parent data[0] = 0

Child Process
Child PID: 1715
Address of data in child: 0x70bd5b02f010
Before modification: data[0] = 0
After modification: data[0] = 9999
Child modified the memory.
Copy-on-Write creates a private copy of the modified page.
After child modification, parent data[0] = 0
Parent still has its original data.

sirichandana@siri:~/practical_memory$ █
```